Topology Optimization
Lecture 1

Introduction to the course

Math fundaments:
Function and vector spaces
Interpolation
L^2 projection

Course Formalities

- 5 ECTS
- 1 mandatory hand-in needed to be approved to be allowed to hand in the exam assignment. Mandatory hand-in is on Brightspace near the middle of the semester.
- Exam: Individual take-home assignment.
  Exam hand-in is on WiseFlow at the end of the semester.
- Lectures and Exercises each week.
- We will use different books and copies:
  - Copies on Brightspace from:
    The Finite Element Method: Theory, Implementation, and Applications. Mats G. Larson and Fredrik Bengzon. ISBN 978-3-642-33287-6.
  - Free book online:
    Solving PDEs in Python, The FEniCS Tutorial I. Hans Petter Langtangen and Anders Logg. ISBN 978-3-319-52462-7. https://www.springer.com/gp/book/9783319524610
  - Copies on Brightspace from:
    Introduction to Optimum Design, 4th edition, J. S. Arora, ISBN: 978-0-12-800806-5
    Numerical Optimization Chapter 17, J. Nocedal & S. J. Wright, https://doi.org/10.1007/978-0-387-40065-5
  - Various papers on topology optimization.
  - See the complete list on Brightspace….

PDE based Mathematical Modeling

- Partial differential equations are mathematical equations used to model many different physical systems.
- PDE based mathematical modeling deals with setting up and solving partial differential equations for different physical systems, e.g.
  - Solid Mechanics
  - Heat Transfer
  - Fluid Mechanics
  - Electromagnetic Wave Propagation
  - Magnetism
  - …
- When the solution can be computed, optimization methods may be used to improve the performance of the physical system.
  - The performance is measured by calculating an appropriate quantity using the solution.
  - We could (and will!) for example calculate the stiffness and try to maximize it.
- In this course we will consider some different PDEs, which often come up in mechanical engineering applications.

Finite Element Method (FEM)

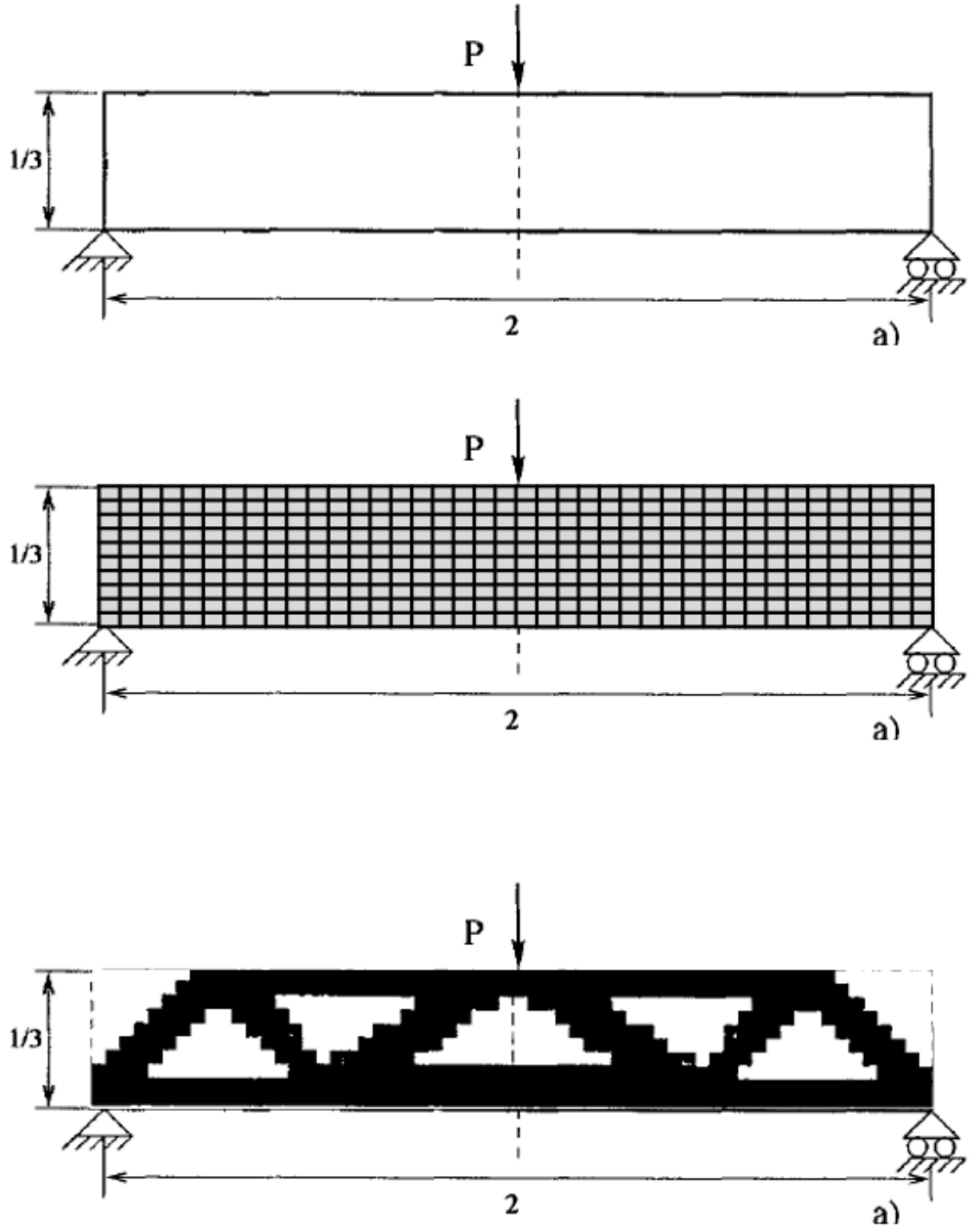
- In some situations, the PDEs may be solved using analytical methods.
- Unfortunately, for many interesting problems the PDEs become so complicated that an analytical solution is not possible to obtain.
- Numerical methods are then used instead.
- The finite element method is one such method, which is extremely good for solving PDEs with complicated geometries.
- Many other numerical methods exist for PDEs.
- The analytical solutions can be extremely important for analysis and understanding, but also for testing numerical methods.
- In this course we will solve different PDEs using the Finite Element Method in the same unified mathematical framework, FEniCS 2019, in Python 3.
  - Everything is Open Source!
  - There is a newer version named FEniCSx, but we will NOT use this.
  - Install version 2019.1 or 2019.2 as described in the installation file on Brightspace.

Topology Optimization

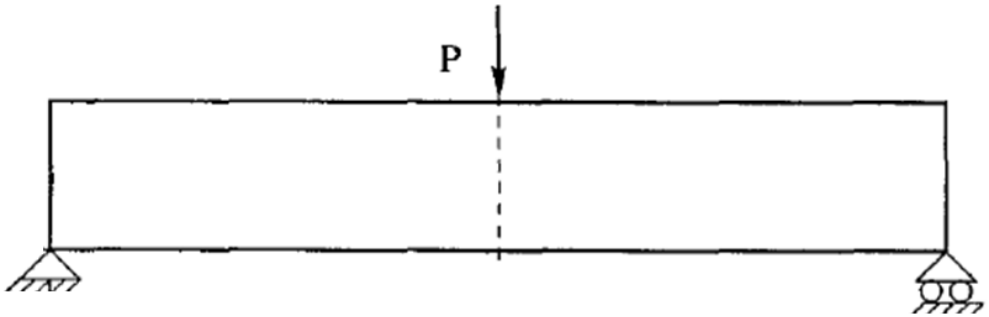
- Topology Optimization deals with automatically calculating optimized geometries.
- This is often done by combining PDEs, FEM and optimization algorithms.
- In this course we will combing exactly this and investigate different techniques, which are often used in topology optimization
  - Adjoint sensitivity analysis.
  - Filtering and Projection.
  - Robust Topology Optimization.
  - (Linear) Equation Solvers.
  - Parallel Programming.
- Although we use a specific framework in this course, one can use the same techniques in many other numerical packages, e.g.
  - COMSOL
  - Ansys
  - OpenFOAM
  - MATLAB
  - …

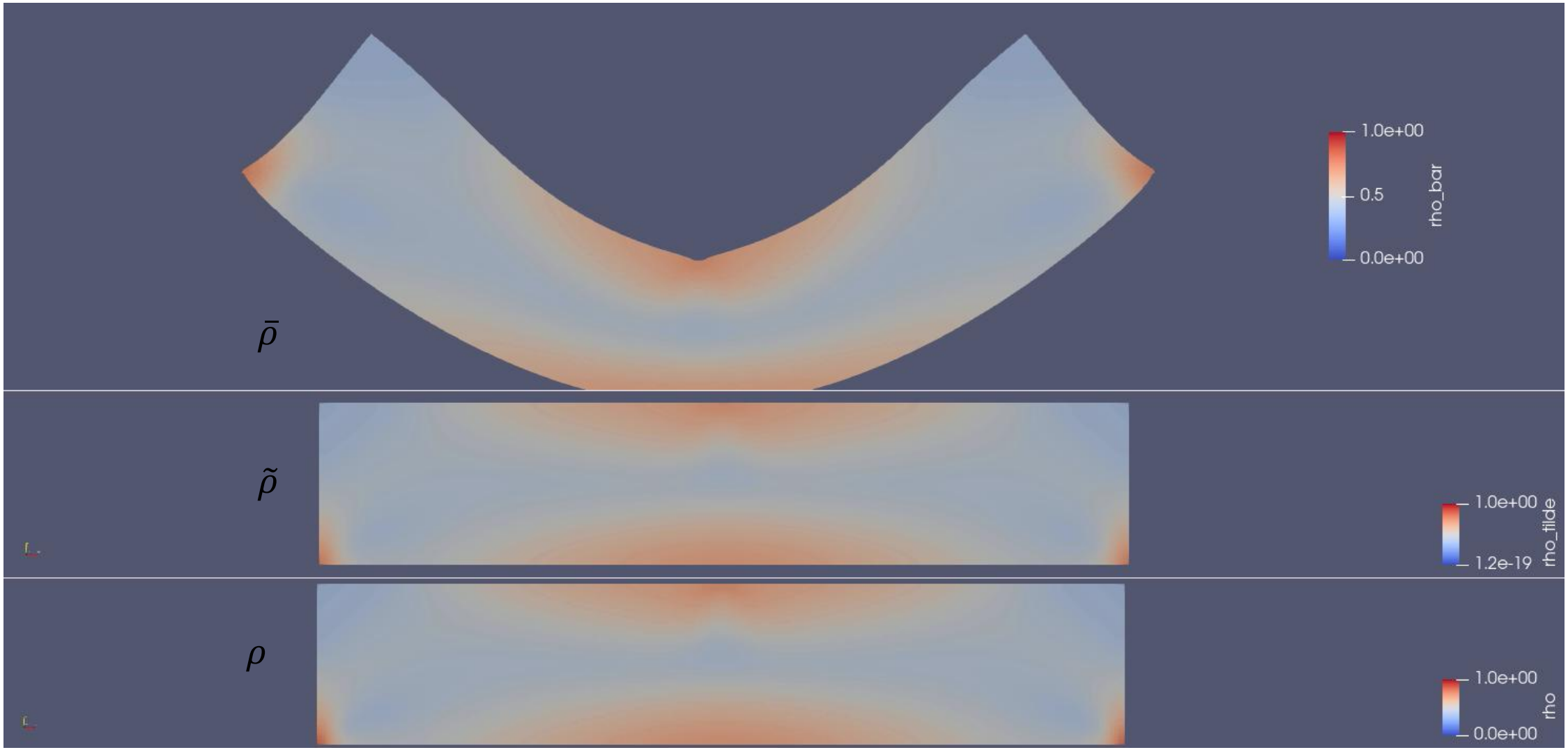
The Basic Idea

- In topology optimization, one defines a design area or volume, named the design space.
- An area is used for 2D problems.
- For 3D problems, a volume is used.
- This design space is divided into elements (or pixels or voxels).
- Each element of the design space can either contain material or be empty.
- An empty element has a “density” of 0 and an element containing material has a “density” of 1.
- An objective function is minimized or maximized with respect to the element densities, acting as “design variables”.
  - Example: Maximize the stiffness.



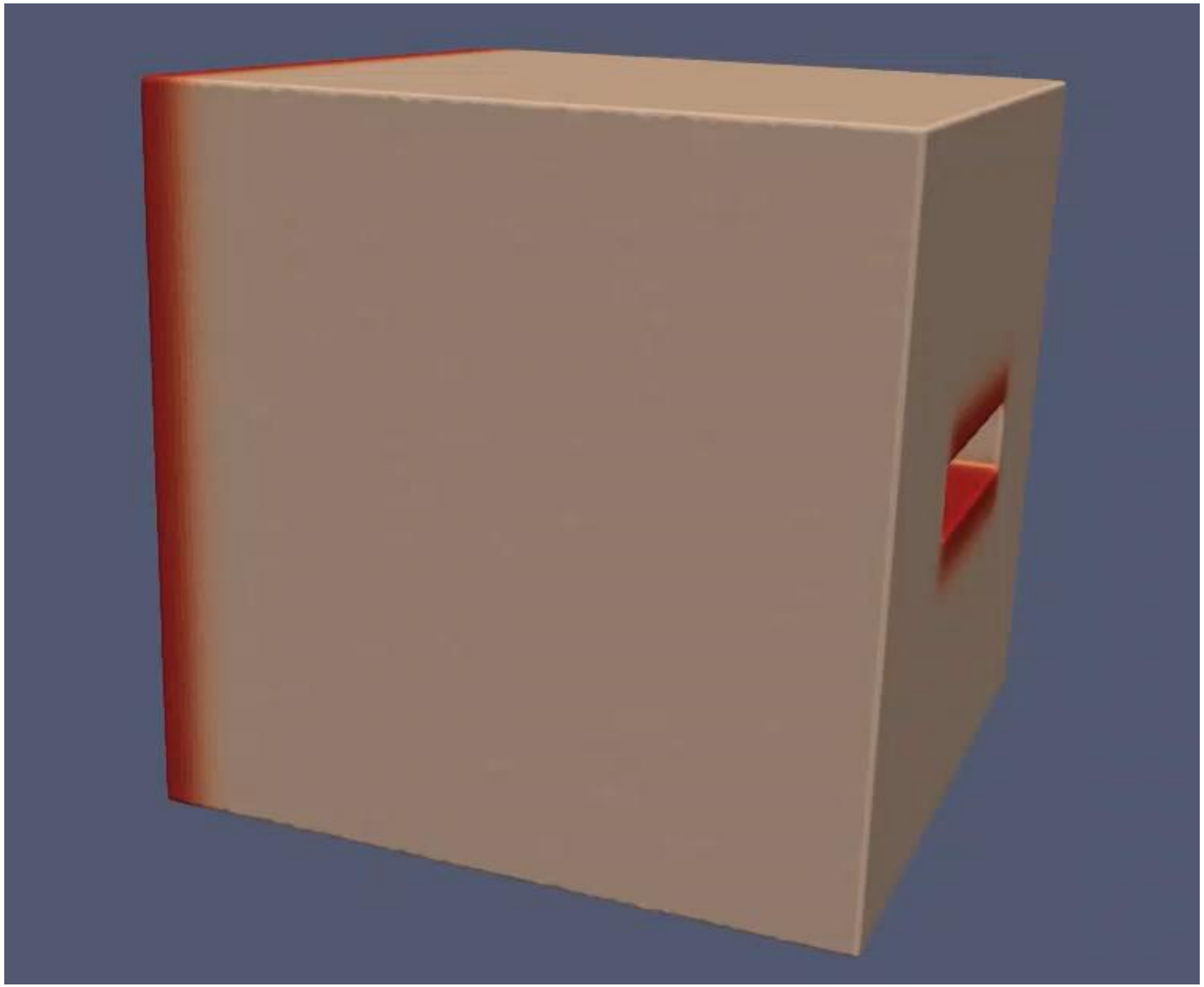
The most used example: MBB Beam



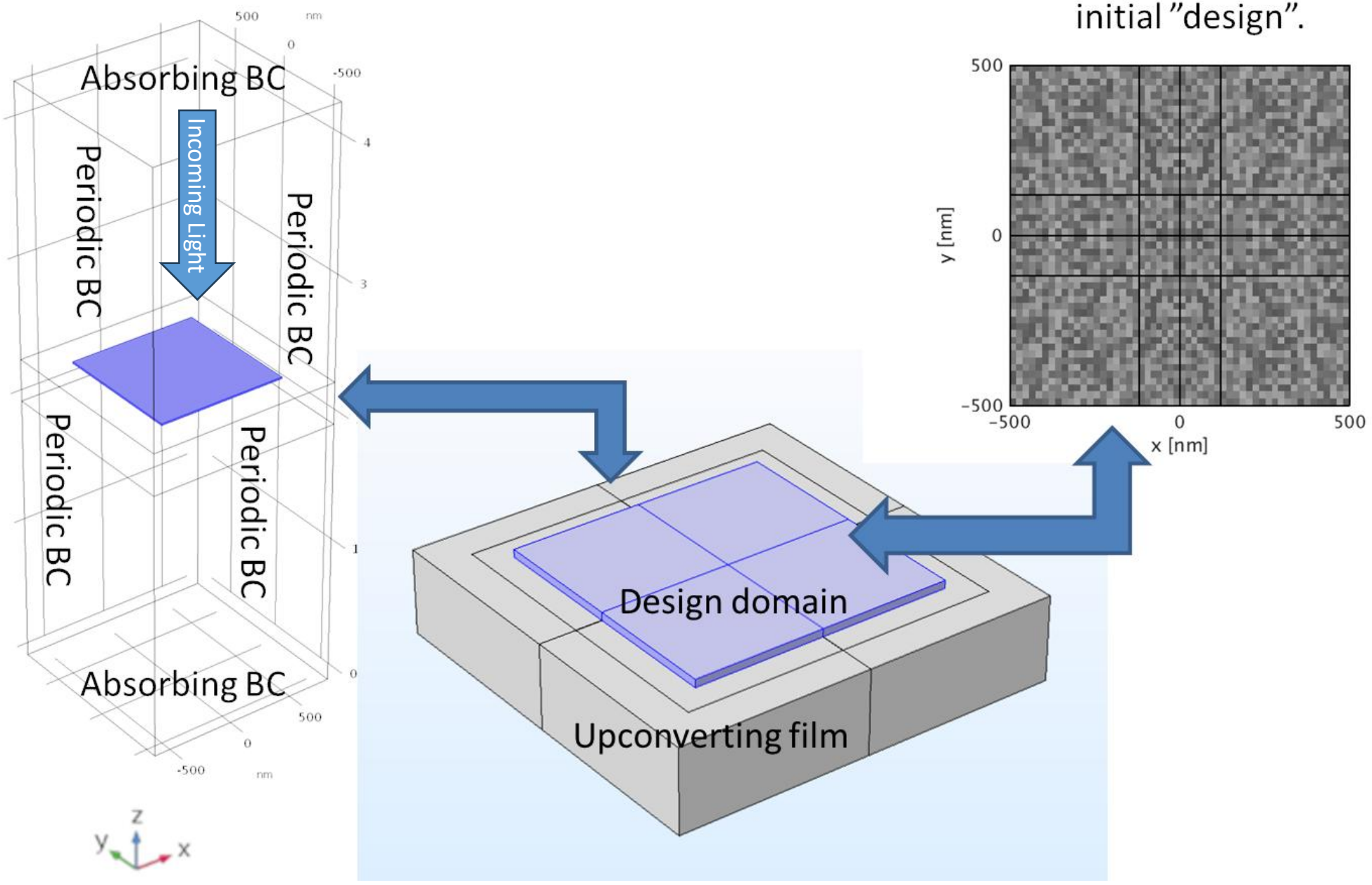


Compliant Mechanism Example

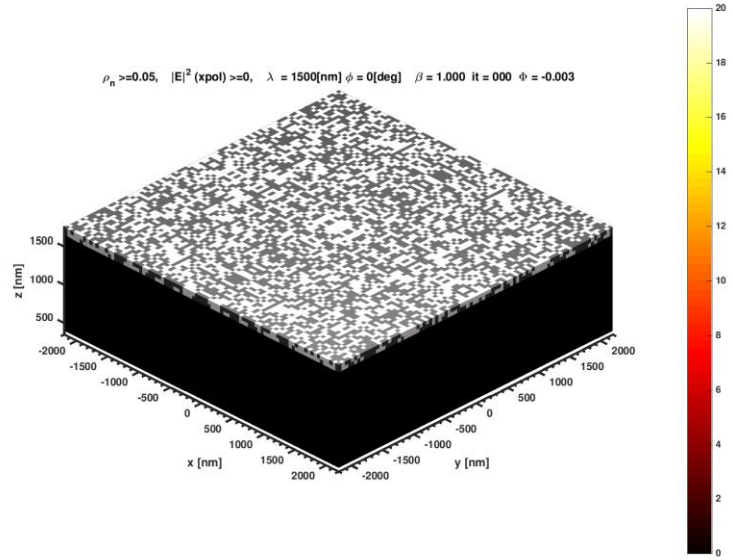
- 3D compliant mechanism gripper.
- Optimized for maximum “grip” displacement.
- Millions of design variables.
- “Robust” optimization utilized.
- Parallelized and run on multiple cores to minimize “design time”.

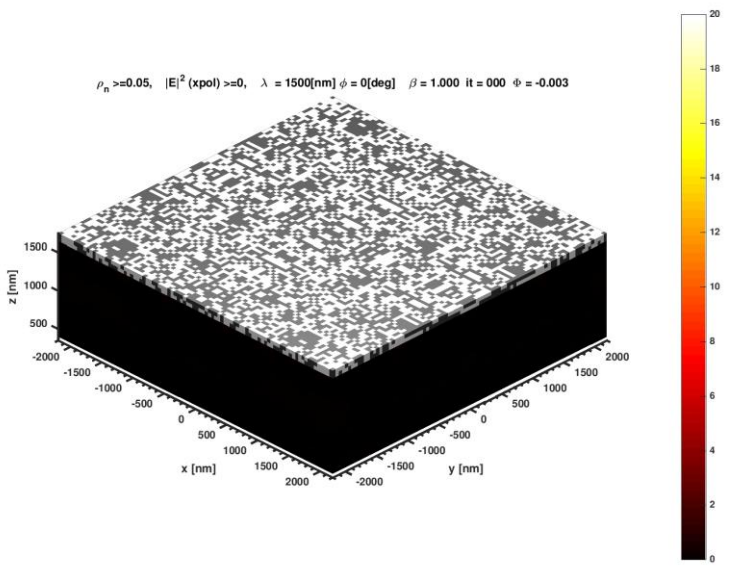


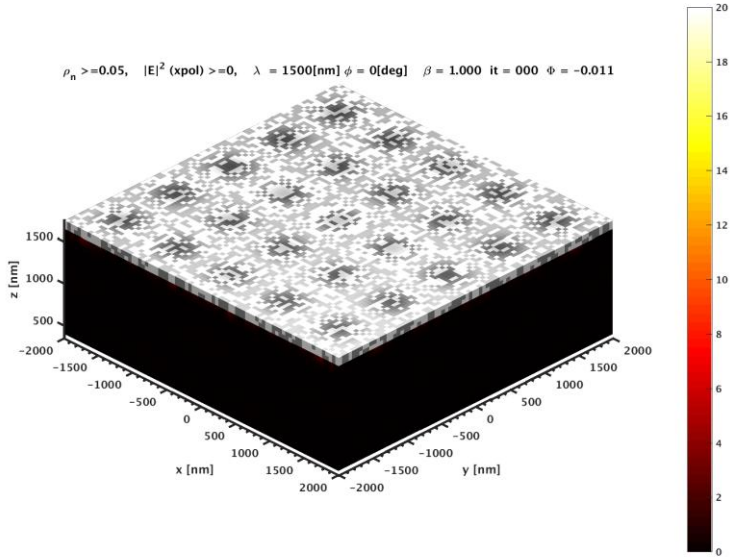
Design of a nanosurface for light concentration

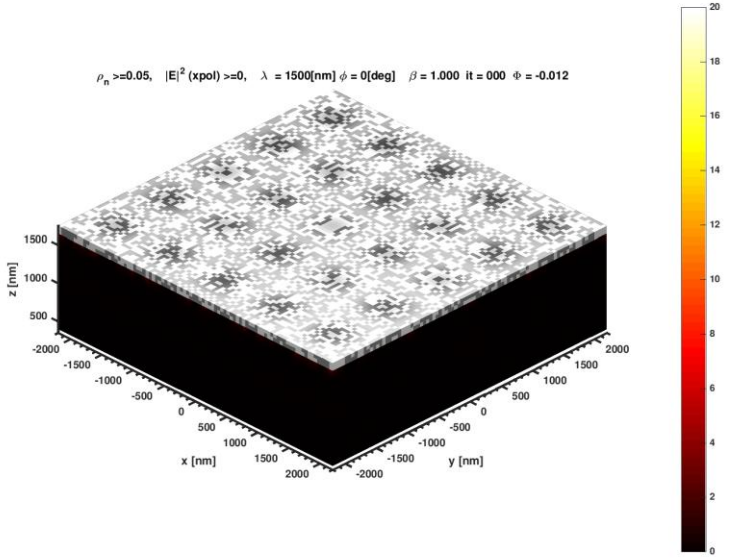


Design of a nanosurface for light concentration



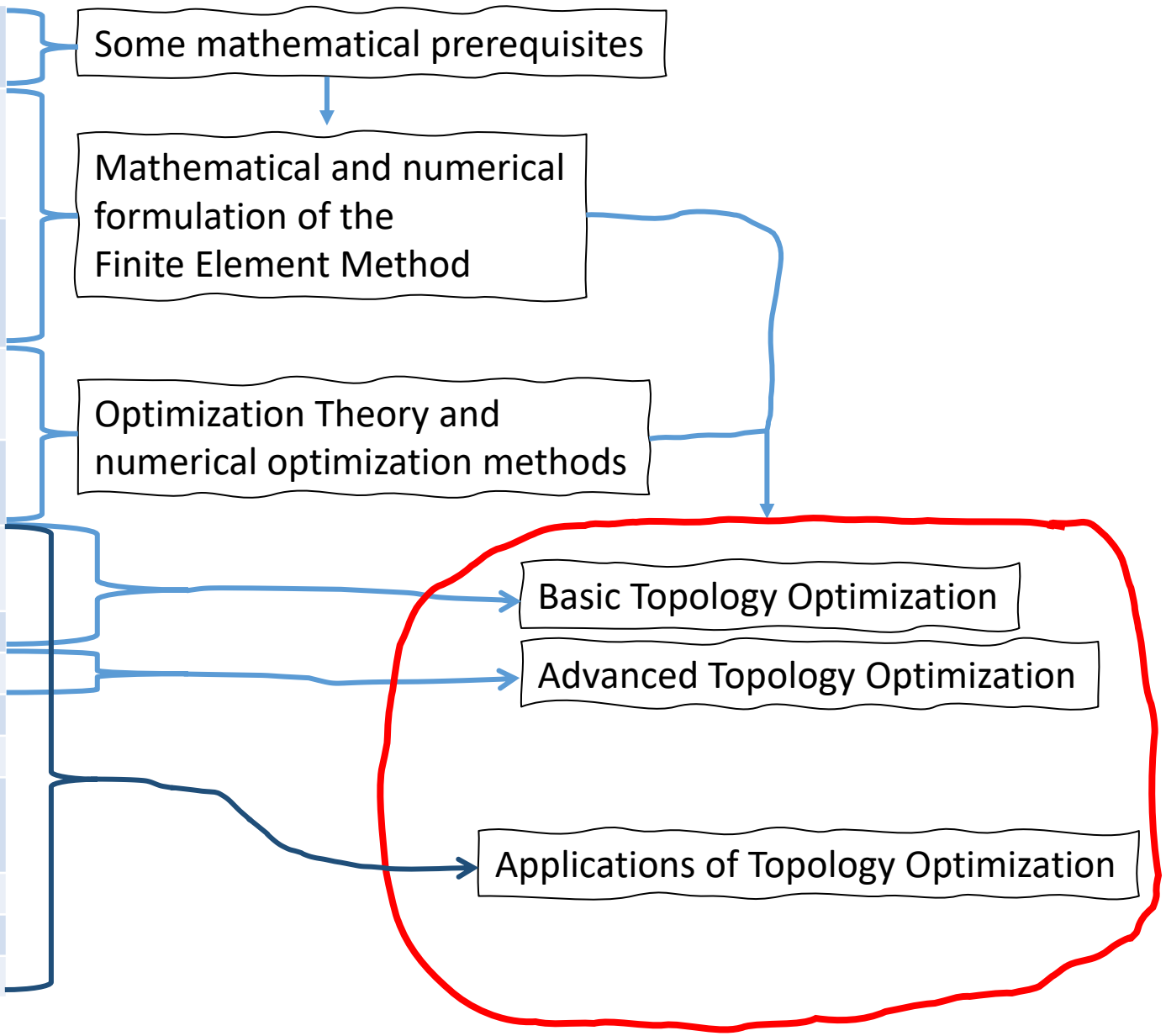






Course overview

No.	Topics
1	Introduction Function and vector spaces, Interpolation, L^2 projection
2	The Finite Element Method for PDEs 1: Poisson's Equation, The Finite Element Method, Properties of the Finite Element Method
3	The Finite Element Method for PDEs 2: Index notation, Linear Elasticity, Non-linear Poisson's equation, Non-linear Elasticity
4	Constrained Optimization 1: Optimality, KKT Theorem, Numerical Optimization Algorithms
5	Constrained Optimization 2: Method of Moving Asymptotes, MPI and PETSc
6	Introduction to Topology Optimization: The idea, Filtering and Projection, Python implementation
7	Gradients
8	Robust topology optimization, More on MPI
9	Wave equations
10	Acoustic waves and more on Robust Optimization
11	Penalty and Augmented Lagrangian Methods Stress-constraints in Topology Optimization 1
12	Stress-constraints in Topology Optimization 2
13	Light concentration using nanoparticle design
14	Heat conduction



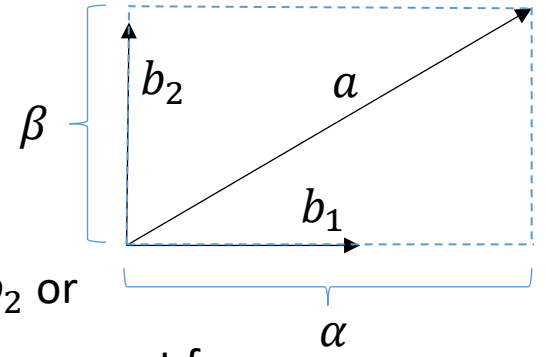
1D vs 2D vs 3D

- We live in a 3+1-dimensional world.
- In many cases, we need only to solve static problems.
- But for some problems we need to consider time-dependence (the “+1”), which is most often treated differently than the spatial dependence.
- Time-dependent problems requires one to solve a series of problems.
- Doing numerical simulations in 3+1 dimensions is generally quite demanding on
  - The computer, when doing the computations.
  - The lecturer, when drawing the figures.
  - The student, to grasp all the mathematical details.
- We will try to get away with discussing the problems mainly in only 1D and 2D and keep them static to make it as simple as possible.
  - 3D and time-dependence can also be done in the same formalism.
  - Sometimes things can be a bit different in 3D and (3+1)D, though…

FEM for Partial Differential Equations

- The first topic of this course is to learn and understand FEM for PDEs.
- The short description of FEM in MATLABs documentation says:
  - “FEM can be summarized in the following sentence: *Project the weak form of the differential equation onto a finite-dimensional function space*.“
- In the next few weeks, we will learn exactly what this means and how to use it to derive Finite Element Models.
- But we need to start with something simpler: Interpolation.
- We do this mainly in 2D as 1D is too simple and 3D is too complicated
  - The basic principles stays the same in all dimensions!
- Some math is needed that we will now quickly review. Most concepts and ideas should already be known to you…

Vector and Function Spaces



$a = \alpha b_1 + \beta b_2$ or

$a = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$ in component form

Some write the vector "a" as a, some use $\vec{a}$, some use $\boldsymbol{a}$…

- A function space is a collection of functions where
  - a set of basis functions are defined.
  - all functions in the given function space can be made from a linear combination of the basis functions.
- A function space is a type of vector space, so it follows the same rules.
- A vector space: Let $f_1, f_2, f_3 \in$ V and c , $r \in \mathbb{C}$ then
  - Addition $f_1+f_2=f_3 \in$ V.
    - Commutativity: $f_1+f_2=f_2+f_1$.
    - Associativity: $(f_1+f_2)+f_3= f_1+(f_2+f_3)$.
    - Identity: $f_1+0=0+f_1=f_1$
  - Scalar multiplication $cf_1=f_2 \in$ V.
    - Associativity: $c(rf_1)=r(cf_1)$.
    - Distributivity over scalars: $(c+r)f_1=cf_1+rf_1$.
    - Distributivity over vectors: $c(f_1+f_2)=cf_1+cf_2$
    - Identity: $1f_2=f_2 1=f_2$.

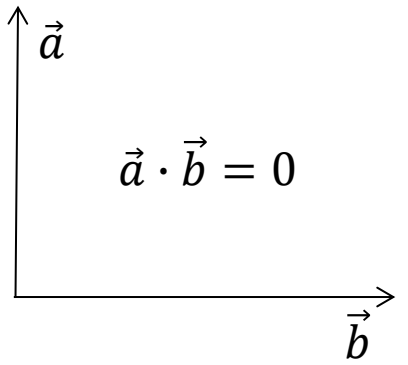
Example: Let V be the space of linear functions,
$f \in$ V => $f=a+bx$, $a,b \in \mathrm{R}$

$f_1+f_2 = a_1+b_1x + a_2+b_2x= (a_1+a_2)+(b_1+b_2)x= a_3+b_3x \in$ V

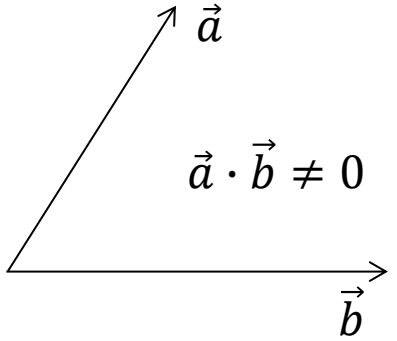
$cf_1 = c(a_1+b_1x)= ca_1+cb_1x= a_2+b_2x \in$ V

Vector and Function Spaces

- A scalar product (inner product) can be defined as $f_1 \cdot f_2 = \langle f_1 | f_2 \rangle = (f_1, f_2)$ (using 3 different notations)
  - $(f_1, f_2 + f_3) = (f_1, f_2) + (f_1, f_3)$
  - (f_1, f_2)=$(f_2, f_1)^*$ (* means complex conjugate)
  - $(f_1, cf_2) = c(f_1, f_2)$ and $(cf_1, f_2) = c^*(f_1, f_2)$
  - If $(f_1, f_2) = 0$ then f_1 and f_2 are said to be *orthogonal*.



- For a function space, the scalar product is often defined as (in 1D with the function defined from x=a to x=b)
  - $(f_1, f_2) = \int_a^b f_1^*(x)\, f_2(x) dx$
    where integration limits depend on the function space considered.
- A vector space with an inner product (scalar product) is named an inner product space.
- A norm is a property that satisfy
  - $\|f_1\| \geq 0$ for any f_1 and $\|f_1\| = 0$ if and only if $f_1 = 0$.
  - $\|cf_1\| = |c|\|f_1\|$ for any complex scalar *c*.
  - $\|f_1 + f_2\| \leq \|f_1\|$+$\|f_2\|$. This is known as the triangle inequality.



- Commonly used norms:
  - $\|f_1\|_{L^2} = \sqrt{f_1 \cdot f_1} = \langle f_1 | f_1 \rangle^{1/2} = \left(\int_a^b f_1^*(x) f_1(x) dx \right)^{1/2} = \left(\int_a^b |f_1(x)|^2 dx \right)^{1/2}$, the L^2 norm.
  - $\|f_1\|_p$=$\left(\int_a^b |f_1(x)|^p dx \right)^{1/p}$, the p-norm.
  - $\|f_1\|_{\max} = \|f_1\|_\infty = \max_i \left((f_1)_i \right) = \lim_{p \to \infty} \|f_1\|_p$, the max-norm or infinity norm. Often used in numerical calculations with "big enough *p*".
- A vector space with a norm is named a *normed vector space*.

Vector and Function Spaces

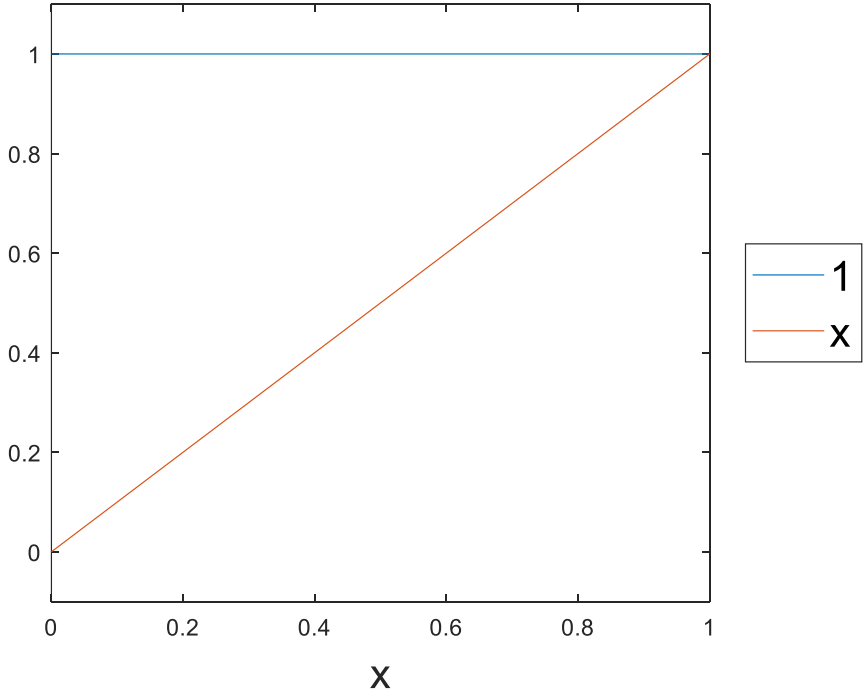
- When a vector space is said to be *complete* then for *any* $f \in$ V, there exist *N* basis vectors φ_i together with *N* scalars c_i such that

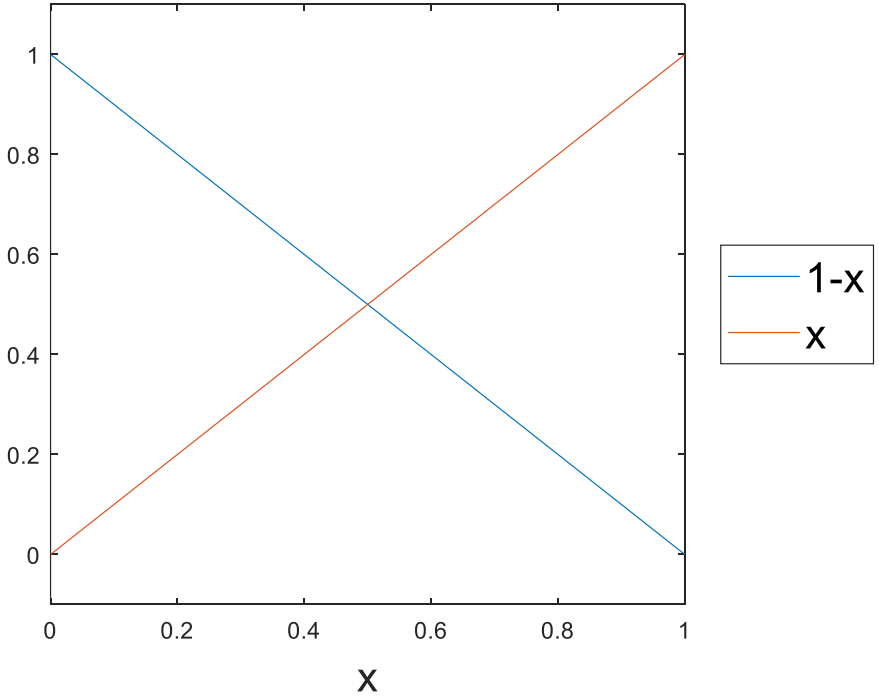
  $$f = \sum_{i=1}^{N} c_i \varphi_i$$

  N is the dimension of the vector space.
  N can be both finite and infinite.
  - *N* will (later) be related to the number of degrees of freedom in a Finite Element calculation.
  - c_i can (later) be interpreted as, e.g. , nodal displacements in a Finite Element calculation.
  - φ_i will (later) be interpreted as shape functions in a Finite Element calculation.
- If $\left(\varphi_i, \varphi_j\right) = 0$ for all i, j, the basis is orthogonal.
  - If *additionally,* $(\varphi_i, \varphi_i) = 1$ for all i the basis is said to be orthonormal.
- A complete inner product space with a L_2 norm is known as a Hilbert space.
- Vector and function spaces are thus very well-defined mathematical objects and can be used to prove many interesting things about functions and numerical methods.
- We will see a bit of this later.
- The most important concept is that we can think of functions in the same way as we think about vectors, and we can expand complicated function into a series of much simpler basis functions.
- The goal of a numerical method is often to determine the unknown expansion coefficients, c_i, given a vector space with known basis functions.

Vector and Function Spaces

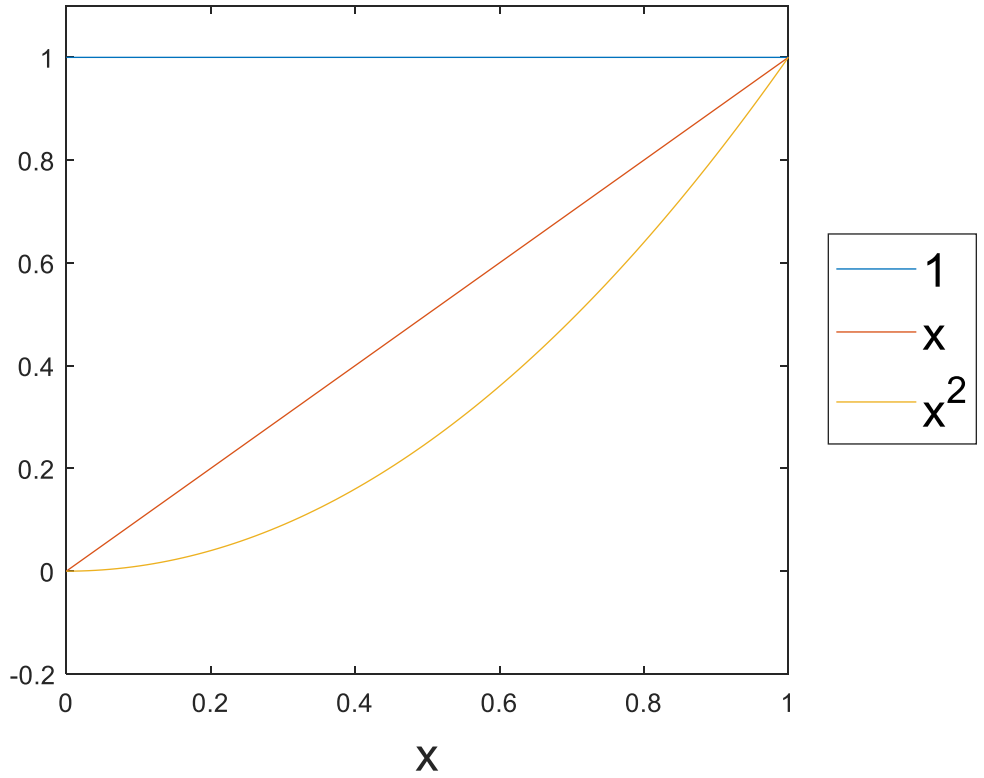
- Example: P_1: The set of linear real functions in 1D:
  - Functions are on the form $f(x)=c_1+c_2x$.
    - Example: $f(x)=2+4x$, $c_1=2$, $c_2=4$.
  - c_1 and c_2 are real numbers and x is the argument.
  - The set of functions {1,x} are the *canonical* basis functions.
  - We can work with the functions or with the vectors on component form (c_1,c_2).
  - It can be very handy to change to another basis, e.g. {1-x,x}.
    This one is named a nodal basis, with nodes at x=0 and x=1.
    - Example: Example: $f(x)=2+4x=2(1-x)+6x$ => $c_1=2$, $c_2=6$ or (2,6) in the nodal basis. Note $f(0)=2=c_1$, $f(1)=c_2=6$.
  - A basis should be linearly independent, such that $c_1(1-x)+c_2x=0$ for all x if and only if $c_1=c_2=0$.

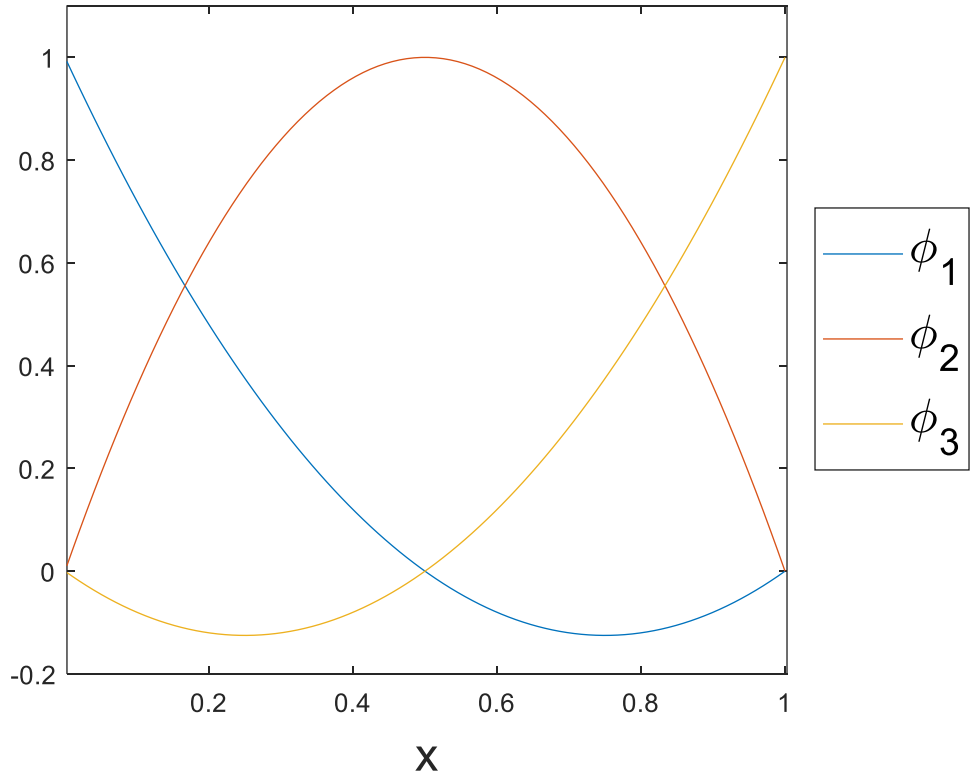




Vector and Function Spaces

- Example: P_2: The set of quadratic real functions in 1D:
  - Functions are on the form $f(x)=c_1+c_2x+c_3x^2$.
  - c_1, c_2 and c_3 are real numbers and x is the argument.
  - The set of functions $\{1,x,x^2\}$ are the *canonical* basis functions.
    - Example: (2,-3,1) in the canonical basis => $f(x)=2-3x+x^2$
  - We can work with the functions or with the vectors on component form (c_1,c_2,c_3).
  - The components (c_1,c_2,c_3) only make sense if we know which basis is associated with the components.
  - Example of another basis: $\{1-3x+2x^2,4x-4x^2,-x+2x^2\}=\{\phi_1,\phi_2,\phi_3\}$ (a nodal basis with nodes x=0, x=1/2, x=1).
    - Example: (2,3/4,0) in the nodal basis => $f(x)= 2(1-3x+2x^2)+3(4x-4x^2)/4+0(-x+2x^2)=2-3x+x^2$.





Space of linear functions in 2D

The space of linear functions on a triangle K is given by

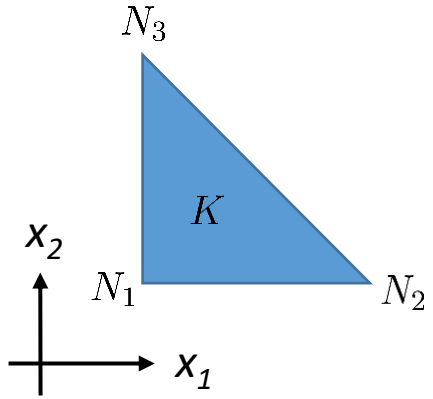
$$P_1(K) = \{v \mid v(x_1, x_2) = c_0 + c_1 x_1 + c_2 x_2 \ , \ (x_1, x_2) \in K \ , \ c_i \in \mathbb{R}\}$$

There are 3 constants (c_0, c_1 and c_2) to specify a given function.

The constants can be determined using the function values at 3 different points.

Imagine we know the values of a linear function at the nodes of a triangle, $v(N_1) = v_1$, $v(N_2) = v_2$ and $v(N_3) = v_3$ and want to determine the c_is.
We can write down what we know:

$$\begin{aligned}
v(N_1) = v_1 &= c_0 + c_1 x_1^{(1)} + c_2 x_2^{(1)} \ , \ N_1 = (x_1^{(1)}, x_2^{(1)}) \\
v(N_2) = v_2 &= c_0 + c_1 x_1^{(2)} + c_2 x_2^{(2)} \ , \ N_2 = (x_1^{(2)}, x_2^{(2)}) \\
v(N_3) = v_3 &= c_0 + c_1 x_1^{(3)} + c_2 x_2^{(3)} \ , \ N_3 = (x_1^{(3)}, x_2^{(3)})
\end{aligned}$$



By solving the 3 equations for the 3 c_is, we have determined v in the canonical basis, $\{1, x_1, x_2\}$.

Space of linear functions in 2D

The nodal basis functions $\lambda_i(x_1, x_2)$ are defined by

$$\lambda_i(N_j) = \delta_{ij} = \begin{cases} 1 & , \quad i = j \\ 0 & , \quad i \neq j \end{cases} \quad , \; N_j = (x_1^j, x_2^j) \text{ is the set of coordinates for node } j.$$

(δ_{ij} is named the Kronecker delta function)
The nodal basis function λ_i is 1 at node i and zero at the two other nodes of the triangle.

The nodal basis is often much more convenient than the canonical basis when working with meshes.

A single nodal basis function can be found by solving the 3 eqs. with 3 unknowns for c_0, c_1, and c_2, shown on the previous slide.

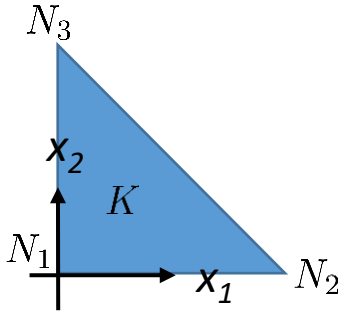
This must be done for each λ_i, $i = 1, 2, 3$, to specify the complete basis.

For the simple triangle with nodes $N_1 = (0, 0)$, $N_2 = (1, 0)$, $N_3 = (0, 1)$ one gets (after solving 3 eqs. with 3 unknows 3 times):

$$\lambda_1 = 1 - x_1 - x_2 \quad , \quad \lambda_2 = x_1 \quad , \quad \lambda_3 = x_2$$

Using in the nodal basis, the linear function over the trinagle is written as

$$v(x_1, x_2) = v_1\lambda_1(x_1, x_2) + v_2\lambda_2(x_1, x_2) + v_3\lambda_3(x_1, x_2)$$

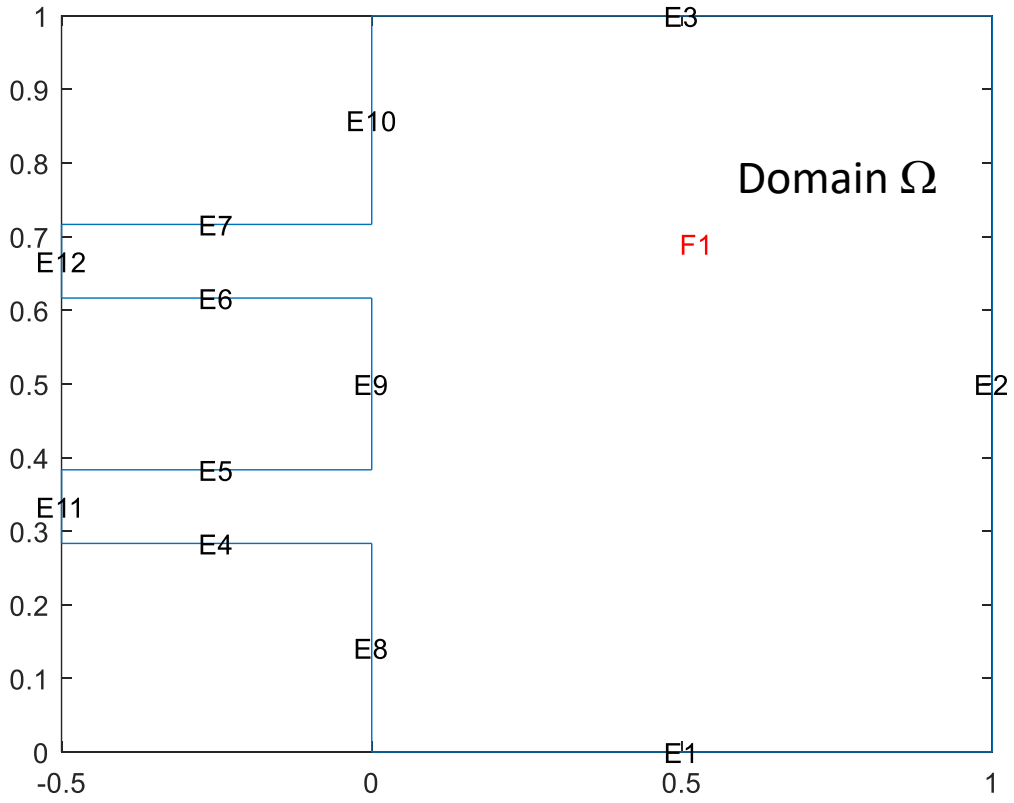


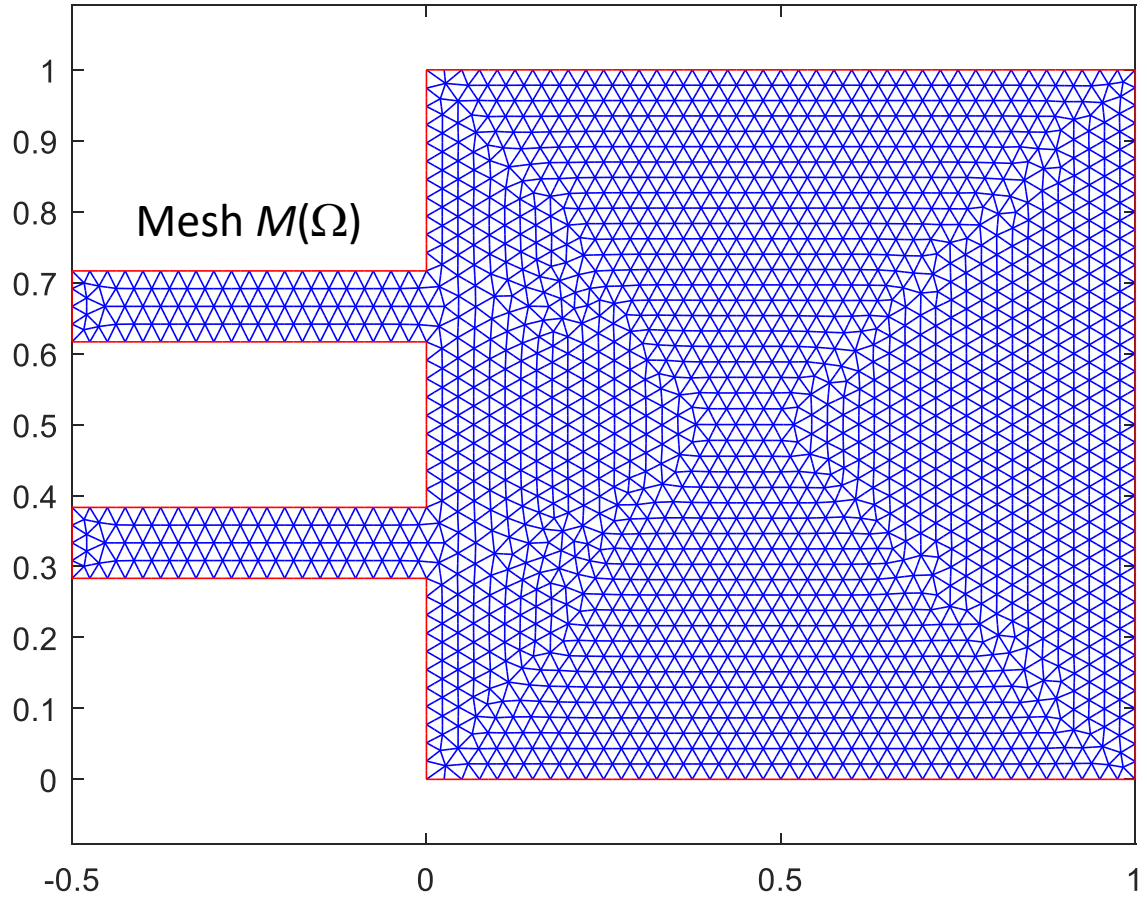
Now, take a minute to show that:

$$\begin{aligned} v(N_1) &= v_1\lambda_1(N_1) + v_2\lambda_2(N_1) + v_3\lambda_3(N_1) = v_1 \\ v(N_2) &= v_1\lambda_1(N_2) + v_2\lambda_2(N_2) + v_3\lambda_3(N_2) = v_2 \\ v(N_3) &= v_1\lambda_1(N_3) + v_2\lambda_2(N_3) + v_3\lambda_3(N_3) = v_3. \end{aligned}$$

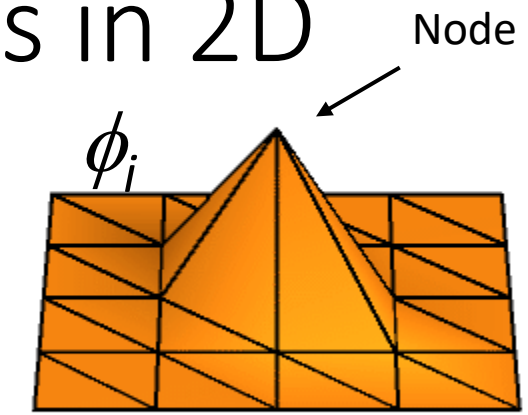
Space of piecewise linear functions in 2D

A domain Ω can be divided into triangles such that $M(\Omega) = \cup_{i=1}^{n_t} K_i \approx \Omega$, where K_i is triangle i in the mesh and the number of triangles is n_t.
Each triangle in the mesh has 3 nodes (or corners or vertices), and the total number of nodes in the mesh is denoted by n_p.





Space of piecewise linear functions in 2D



Let there be n_p number of nodes in the mesh.

Hat functions function are defined over the whole mesh of Ω and linear within each triangle. They are defined by:

$$\phi_i(N_j) = \delta_{ij} \quad , \quad i, j = 1, ..., n_p$$

The hat function ϕ_i is "centered" on node i and non-zero **only** on triangles containing node i.

The space of piecewise linear functions over the mesh of Ω is given by

$$P_1(\Omega) = \left\{ u \;\middle|\; u(x_1, x_2) = \sum_{i=1}^{n_p} u_i \phi_i(x_1, x_2) \ , \ (x_1, x_2) \in M(\Omega) \right\}$$

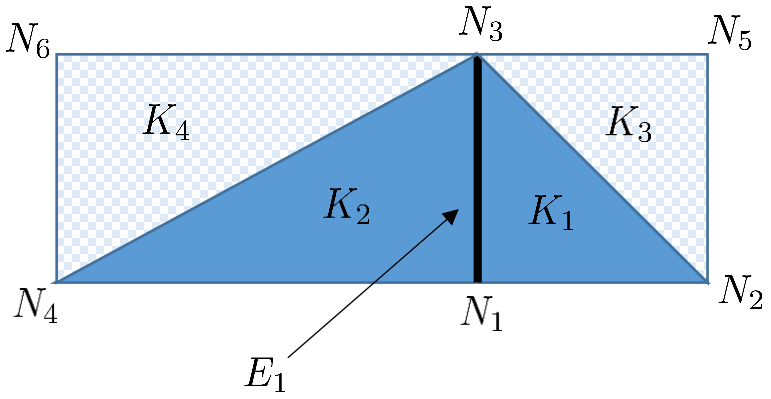
The functions have nodal values $u_i = u(N_i)$.

Space of piecewise linear functions in 2D

The functions have nodal values $u_i = u(N_i)$.

Since a linear function of one variable is completely determined by its value at two points, the functions in $P_1(\Omega)$ will be piecewise continuous.

Mesh consists of:
4 triangles.
6 nodes.
9 edges.



$$P_1(\Omega) = \left\{ u \;\middle|\; u(x_1, x_2) = \sum_{i=1}^{n_p} u_i \phi_i(x_1, x_2) \ , \ (x_1, x_2) \in M(\Omega) \right\}$$

Let the function value be given by $v_1(x_1, x_2)$ in K_1 and by $v_2(x_1, x_2)$ in K_2.

Since both v_1 and v_2 are specified by the nodal values of the function, then $v_1(N_1) = v_2(N_1)$ and $v_1(N_3) = v_2(N_3)$.

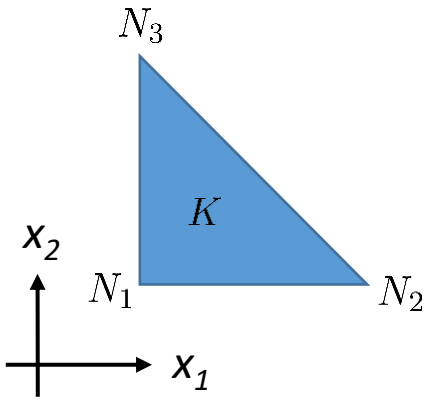
Along the edge E_1, the function is linear. The two nodal values is enough the specify this linear function completely, so $v_1|_{E_1} = v_2|_{E_2}$ and the whole function is piecewise continuous.

The derivative of the function in P_1 will be piecewise constant, $\frac{\partial u}{\partial x_1} \notin P_1$, and thus not continuous over the edges.

Linear Interpolation on a triangle

- For a given function $f(x_1, x_2)$, we can construct the linear interpolant, named πf, over a triangle *K* by evaluating the function f at the nodal points:

$$\pi f = \sum_{i=1}^{3} f(N_i)\lambda_i(x_1, x_2)$$



- A measure of the error of the interpolation is the L^2 norm of the difference between the exact function and the interpolant:

$$\|\pi f - f\|_{L^2(K)} = \left((\pi f - f, \pi f - f)_K\right)^{1/2} = \left(\int_K |\pi f - f|^2 dA\right)^{1/2}$$

- It can be shown that the error satisfy

$$\|\pi f - f\|_{L^2(K)} \leq C h_K^2 \|D^2 f\|_{L^2(K)}$$

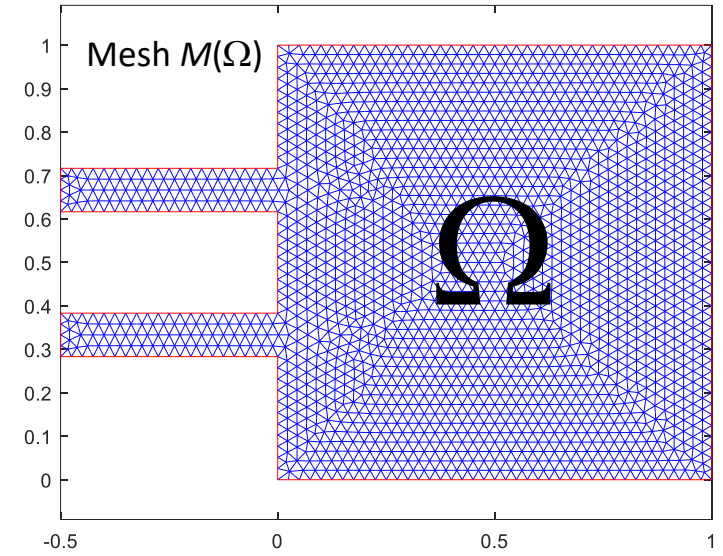
with $D^2 f = \left(\left|\frac{\partial^2 f}{\partial x_1^2}\right| + 2\left|\frac{\partial^2 f}{\partial x_1 \partial x_2}\right| + \left|\frac{\partial^2 f}{\partial x_2^2}\right|\right)^{1/2}$ and
h_K the longest side in the triangle and
$C \propto \frac{1}{\sin \theta_K}$ where θ_K is the smallest angle in the triangle.

- The error of the interpolation thus depend upon:
  - How fast the function "wiggles" $(D^2 f)$.
  - The size of the triangle (h_K).
  - The "stretching" of the triangle (θ_K).

Linear Interpolation on a mesh of triangles

- For a given function $f(x_1, x_2)$, we can construct the linear interpolant over a domain Ω using a mesh of triangles *M*(Ω) by evaluating the function at the nodal points of the mesh:

$$\pi f = \sum_{i=1}^{n_p} f(N_i)\phi_i(x_1, x_2)$$



- A measure of the error of the interpolation is

$$\|\pi f - f\|_{L^2(\Omega)} = \left((\pi f - f, \pi f - f)_\Omega\right)^{1/2} = \left(\int_\Omega |\pi f - f|^2 dA\right)^{1/2}$$

- It can be shown that the error satisfy

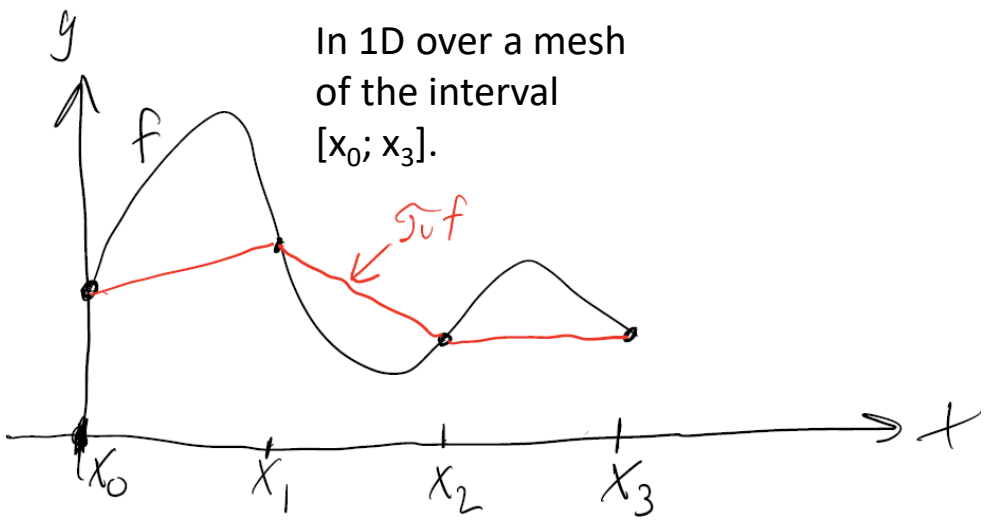
$$\|\pi f - f\|^2_{L^2(\Omega)} = \sum_{K_i \in M(\Omega)} \|\pi f - f\|_{L^2(K_i)} \leq \sum_{K_i \in M(\Omega)} C_{K_i} h^4_{K_i} \|D^2 f\|^2_{L^2(K_i)}$$

with $D^2 f = \left(\left|\frac{\partial^2 f}{\partial x_1^2}\right| + 2\left|\frac{\partial^2 f}{\partial x_1 \partial x_2}\right| + \left|\frac{\partial^2 f}{\partial x_2^2}\right|\right)^{1/2}$ and

h_{K_i} the longest side in triangle K_i and

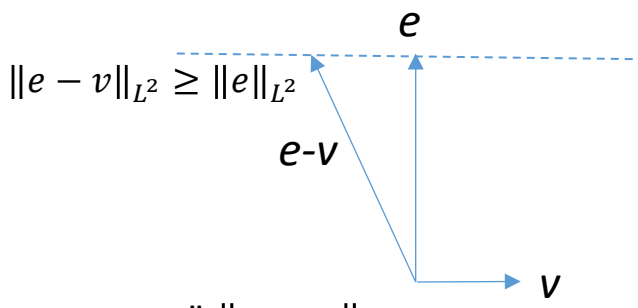
$C_{K_i} \propto \frac{1}{\sin \theta_{K_i}}$ where θ_{K_i} is the smallest angle in triangle K_i.

- The error of the interpolation thus depend upon:
  - How fast the function "wiggles" $(D^2 f)$.
  - The size of the triangles (h_{K_i}).
  - The "stretching" of the triangles (θ_{K_i}).



In 1D over a mesh of the interval $[x_0; x_3]$.

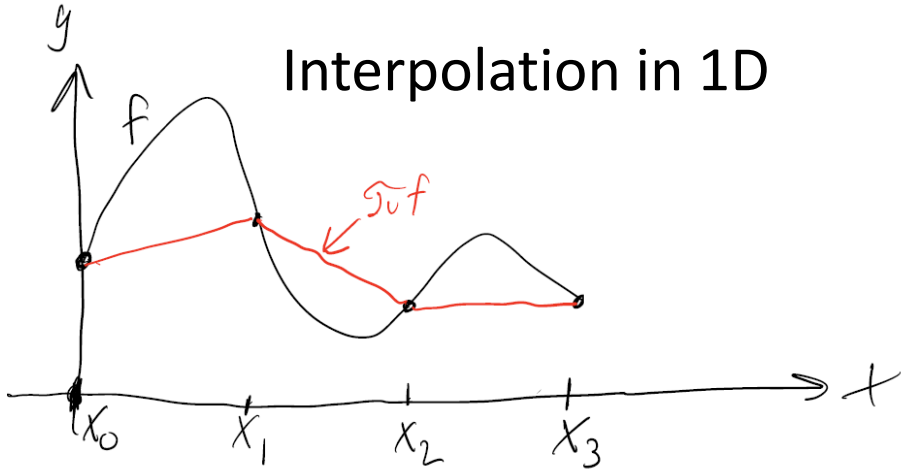
L^2 projection

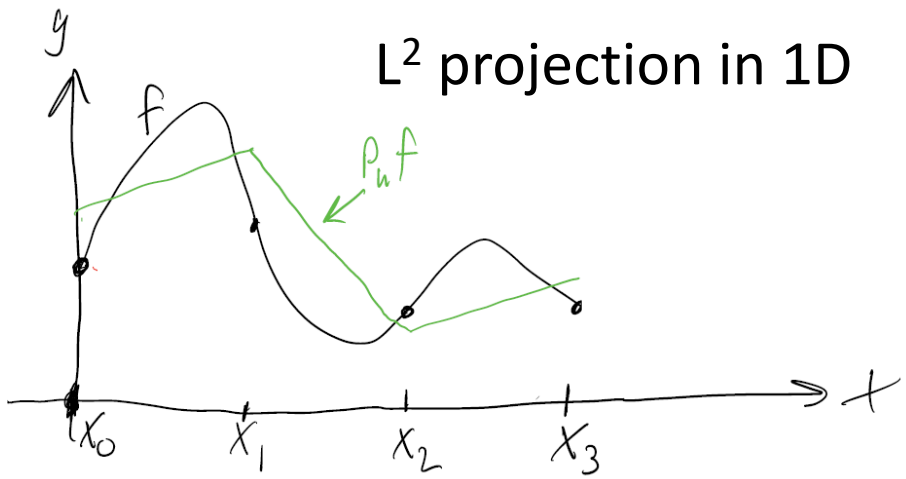


- In interpolation, the error is zero at the nodal points.
- For functions, the L^2 error is the "integrated difference between the function and its approximation".
- When thinking about the functions as vectors, the L^2 error is simply "the length of the difference between two vectors", $\|v - w\|_{L^2}$.
- In a L^2 projection, the objective is to make the L^2 error as small as possible, i.e., solve the optimization problem $\min_{P_h f \in V} \|f - P_h f\|_{L^2}$.
  - If a function (vector) *e* is orthogonal to another function (vector) *v*, then *e* can not be made smaller by adding or subtracting anything in the *v* direction.
  - If the error, $f - P_h f$, can be made orthogonal to "all vectors" in a function space, then it can not be made any smaller within that function space.
- The L^2 projection, named $P_h f$, of a function f onto a function space V is defined implicitly by the property

$$\int_\Omega (f - P_h f) v dA = 0 \quad \text{for all } v \in V$$

- The error $(f - P_h f)$ is orthogonal to all functions in the function space *V*.





L^2 projection

- The L^2 projection, $P_h f$, of f onto a function space V is defined by

$$\int_\Omega (f - P_h f) v dA = 0 \quad \text{for all } v \in V$$

- The goal is to write down *n* equations with *n* unknowns we can solve.
- Since the equation must be true for all $v \in V$, we can write down n_p equations using each of the basis functions ϕ_i of V:

$$\int_\Omega (f - P_h f) \phi_i dA = 0 \quad \text{for } i = 1, \dots, n_p$$

- $P_h f \in V$ so it can be expanded in the basis functions ϕ_i of V using the unknown (degrees of freedom) α_j:

$$P_h f = \sum_{j=1}^{n_p} \alpha_j \phi_j$$

- This is inserted to get (recall that ϕ_i and ϕ_j depend on coordinates x_k ((x_1, x_2) in 2D) and α_j are numbers)

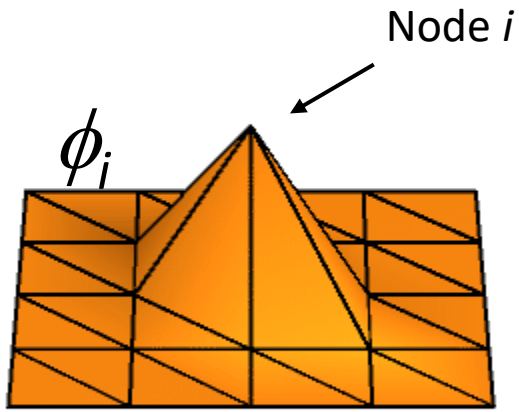
$$\int_\Omega (f - P_h f) \phi_i dA = \int_\Omega \left(f - \sum_{j=1}^{n_p} \alpha_j \phi_j \right) \phi_i dA = 0 \;\Leftrightarrow$$

$$\int_\Omega f \phi_i dA = \int_\Omega \sum_{j=1}^{n_p} \alpha_j \phi_j \, \phi_i dA = \sum_{j=1}^{n_p} \alpha_j \int_\Omega \phi_i \phi_j dA \;\Leftrightarrow\; \sum_{j=1}^{n_p} M_{ij} \alpha_j = b_i \;\text{ also written as }\; M_{ij}\alpha_j \text{ or } M\alpha$$

$M_{ij} = \int_\Omega \phi_i \phi_j dA$ and $b_i = \int_\Omega f \phi_i dA$. M_{ij} is named the mass matrix and b_i is named the load vector.

- The L^2 projection is thus calculated by solving a system of linear equations, $M_{ij}\alpha_j = b_i$.

L^2 projection



- The L^2 projection is thus calculated by solving a system of n_p linear equations with n_p unknowns in α_j:
  - $M_{ij}\alpha_j = b_i$.
  - $b_i = \int_\Omega f\phi_i dA$
  - $M_{ij} = \int_\Omega \phi_i\phi_j dA$ is sparse, since the basis functions are only non-zero near one of the nodes.
  - M_{ij} is symmetric.
- All the integrals can be done numerically and many of them are zero.
- Calculating b_i and M_{ij} by doing the integrals is termed 'assembly' (of the equation system) in Finite Element language.
- The mass matrix is also used in time-dependent PDEs, where it enters, e.g., in the acceleration term in Newton's 2nd law.
- It is quite convenient to do L^2 projections when working with finite element calculations, as you will see throughout the course.